MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE REPUBLIC OF KAZAKHSTAN
Astana IT University

School of Software Engineering
Educational programme: Software Engineering

REPORT

on the approbation period of the diploma project

Topic:

Development of a web platform for online booking in cinemas

Team:
Nurbek Shagarbek (SE-2423)
Ismail Zhanatay (SE-2423)
Erasyl Meirambekov (SE-2423)

Supervisor:
Chingis Kharmysov, Assistant Professor

Repository: [GitHub / GitLab link]
Task tracker: [YouTrack link]

Astana 2026

CONTENTS

1. Project proposal
2. Evidence of relevance
3. Literature review
4. Comparison of existing systems
5. Technology comparison
6. Requirements specification
7. Architecture and diagrams
8. UX prototype
9. Verification and evaluation plan
10. Project management and risks
11. Supervisor feedback
References

1. Project proposal

1.1 Title, team and supervisor

Title: Development of a web platform for online booking in cinemas.

Supervisor: Chingis Kharmysov, Assistant Professor, School of Software Engineering. Weekly meetings with code review are planned.

Table 1 – Team members and responsibilities

Student	Role	Account	Responsibilities
Nurbek Shagarbek	Lead Backend Developer / Software Engineer	Shagarbek	Backend architecture, booking and transaction logic, PostgreSQL, API, tests, technical documentation
Ismail Zhanatay	Back-end developer	ikki1769	Database structure, REST API, temporary seat locking and protection against double booking, authentication (JWT, roles), payment integration, tests
Erasyl Meirambekov	Front-end developer	Exzou0	Web client (Next.js, TypeScript): seat map, booking flow with hold timer, responsive layout, integration with the API

Roles are taken from the students' justification documents; each student must commit under their own account (Section 10). The final back-end stack must be agreed by the team (Section 5, D1).

1.2 Problem statement

Cinema visitors want to see the schedule, choose a seat and obtain a ticket without queuing at the box office. Large services already cover this need for big networks, but a smaller cinema operator needs a platform that it controls itself and that guarantees one essential property: a seat must never be sold to two people. A booking system must handle many users who select the same seats at the same moment, keep temporary reservations while the user pays, and restore the seat to sale if payment is not completed.

The earlier course prototype of this project (Advanced_Final, identified in the report as Exzou0/Advanced_Final) demonstrated the basic flow of movies, users and tickets. Its code and the team-supplied demonstration screenshots provide relevant prior experience (Section 8.2). The reviewed source snapshot mixes session and movie identifiers, contains hard-coded credentials, and duplicates ticket data in a process-local map in addition to PostgreSQL. Protection against concurrent seat conflicts cannot be established without the missing database schema and tests (Section 9.3). The diploma project develops this experience into a verifiable engineering solution.

1.3 Target users

- **Moviegoers** (guests and registered users): browse movies and sessions, select and book seats, view tickets.
- **Cinema administrators:** manage movies, halls, seat layouts and sessions.
- **Cinema managers:** view sales and occupancy statistics.

1.4 Aim and objectives

Aim: to design, implement and evaluate a web platform for online cinema ticket booking whose booking core is demonstrably free of double bookings and meets defined performance, security and usability targets.

Objectives:

1. Collect and document evidence of the problem through 3–5 discussions with user representatives and an analysis of publicly available information about existing services.
2. Review at least 12 sources (at least 8 scientific publications) on concurrency control, architecture and API style for booking systems.
3. Compare existing cinema-ticketing services against a fixed set of criteria and justify the need for the proposed solution.
4. Select and justify the key technologies (backend platform, data storage, seat-hold mechanism, API style, system structure).
5. Specify verifiable functional and non-functional requirements with priorities and acceptance criteria.
6. Design the architecture (context, use-case and draft component diagrams, data model) and UX prototypes of the key scenarios.
7. Implement the backend core (catalogue, authentication, booking with conflict-free seat reservation) and an administrator interface.
8. Evaluate the solution against the baseline prototype using the verification plan (correctness under concurrency, latency, usability).

1.5 Expected outcomes

– A documented analysis (evidence, literature, existing systems, technologies) and a verified requirements specification.
– An architecture description with editable diagrams and UX prototypes.
– A working backend service with a documented REST API, a web client for the main scenarios and an administrator interface.
– Test and measurement results comparing the new solution with the baseline prototype.

1.6 Project scope

Table 2 – Scope

In scope	Out of scope
Online catalogue of movies and sessions	Real money processing and PCI DSS certification
Seat map with real-time availability and temporary seat holds	Native mobile applications
Booking, ticket view and cancellation	Loyalty programme, promo codes, food and drink ordering
Roles: guest, user, administrator, manager	Integration with the box-office hardware of a specific cinema
Administrator tools for movies, halls, sessions	Multi-city, multi-tenant (SaaS) operation
Basic sales and occupancy statistics	
Payment through a sandbox or simulated provider	

1.7 Engineering complexity

The project goes beyond a catalogue application because of four engineering problems: (1) conflict-free concurrent seat reservation, (2) a booking life cycle with expiring holds (held → booked → cancelled / expired), (3) role-based access control with secure credential handling, and (4) measurable non-functional targets verified by concurrency and load tests.

2. Evidence of relevance

2.1 Approach

The requirements call for 3–5 discussions with user representatives or an agreement with a real client. This section therefore separates (a) findings that are already verified from public sources and (b) assumptions that the interviews must confirm or reject. **Interview results must be entered in Table 5 before submission; no results are reported here because they have not yet been collected.**

2.2 Verified findings and assumptions

Table 3 – Status of evidence

Statement	Status	Source
Ticketon.kz is a Kazakhstan-wide aggregator that connects more than 40 cinemas (including Chaplin and Cinemax) and sells e-tickets.	Verified (secondary source)	[S1]
Kinopark launched sales on its own website and announced payments without a commission; Chaplin was reported to be working on its own service.	Verified (secondary source; the article is undated in the retrieved copy)	[S3]
Kinopark's mobile application offers schedules, online ticket purchase, purchase history and reminders.	Verified (store listing)	[S2]
Kino.kz offers a cinema listing and allows users to choose seats and buy tickets online.	Verified (secondary source)	[S4]
The earlier course prototype (Advanced_Final) implements movies, users, JWT authentication and ticket insertion, but protection against two users taking the same seat could not be confirmed in the code.	Verified (own code review)	Repository Exzou0/Advanced_Final
Moviegoers lose time in queues and want to choose seats in advance.	**Assumption** — to be validated in interviews	—
Small independent cinemas need a lightweight, low-cost platform and do not want to depend on an aggregator.	**Assumption** — to be validated in interviews	—
Double booking of a seat is a real operational problem for cinema staff.	**Assumption** — to be validated in interviews	—

2.3 Interview protocol

Semi-structured interviews of 20–30 minutes; participants give consent; notes are stored without personal data in the repository (docs/interviews/). Question guide:

1. How do you currently choose a movie and buy tickets (channel, device, payment method)?
2. What is the most frustrating part of the current process?
3. How important is it to choose a specific seat, and how do you decide which seat?
4. Have you ever found a seat already taken, or two tickets sold for one seat? What happened?
5. How long would you be willing to wait for a payment to be completed before the seat is released?
6. (Staff) How are schedules and prices updated today, and how long does it take?
7. (Staff/manager) Which statistics do you need to see, and how often?
8. What would make you choose a new booking service instead of the one you use now?

Table 4 – Planned respondents

ID	Role	Date	Key findings	Requirements affected
R1	Frequent moviegoer (student)	[name / date]	[to be completed]	[FR / NFR IDs]
R2	Occasional moviegoer (non-student)	[name / date]	[to be completed]	[FR / NFR IDs]
R3	Cinema administrator / cashier	[name / date]	[to be completed]	[FR / NFR IDs]
R4	Cinema manager or owner (potential client)	[name / date]	[to be completed]	[FR / NFR IDs]
R5	Supervisor as academic client (scope and constraints)	[date]	[to be completed]	[FR / NFR IDs]

After the interviews, every assumption in Table 3 is marked as confirmed, rejected or changed, and the priorities in Section 6 are revised accordingly.

3. Literature review

3.1 Method

Sources were selected by searching for concurrency control in booking systems, database isolation, monolithic versus microservice architectures, API styles and backend language performance. Thirteen sources are used (Table 6), of which nine are scientific publications or academic works ([1]–[10], including one repository record [9]). Concurrency-control theory is stable, so three foundational works from 1981–2012 are retained; five of the scientific sources were published in 2022–2025 because architecture and API performance evidence changes quickly. Practitioner material on ticketing system design (blogs and interview guides) was read for orientation only and is not cited as evidence.

3.2 Critical review

Concurrency and consistency. Optimistic concurrency control [1] lets transactions proceed without locks and validates them at commit, which suits low conflict rates; for popular sessions the conflict rate on a few seats is high, so retries may be frequent. The critique of ANSI isolation levels [2] shows that "read the seat, then write the seat" logic can be unsafe under snapshot isolation. PostgreSQL implements serializable snapshot isolation at a moderate cost [3], but a simpler and more transparent protection for this domain is a database constraint that forbids two active tickets for the same seat and session [11]. This is a design inference of the project, to be confirmed by test V-1.

Architecture. Evidence is mixed. A controlled comparison found that the monolith performed better on one machine [5], while an online-ticketing experiment found microservices faster under high load but unstable above 90% CPU [9]; an early case study lists both benefits and operational challenges [4]. For one cinema operator and a three-person team a modular monolith is the lower-risk starting point.

Backend platform and API style. A comparison of API services reports that Go scales better than Node.js as the number of users grows [6]. For API styles, gRPC was fastest in one microservice experiment [7], while REST was more memory-efficient and faster than GraphQL under high load in another [8]; REST is also the native style for browser clients [10].

Gaps and limitations. The studies use generic applications, small request counts or abstract-only availability ([9]); none targets a small-cinema booking platform with expiring seat holds. The project takes this into account by measuring its own workload (Section 9) rather than relying on published numbers.

3.3 Structured analysis of sources

Table 6 – Analysis of publications

No.	Source	Type	Topic and finding	Use in the project	Limitation
[1]	Kung & Robinson, 1981	Journal (ACM TODS)	Optimistic concurrency control: transactions run without locks and are validated at commit.	Establishes the optimistic alternative to locking; basis for version-checked seat updates.	General theory; no evaluation on a booking workload.
[2]	Berenson et al., 1995	Conference (SIGMOD)	Shows that ANSI SQL isolation levels are ambiguous and defines snapshot isolation and its anomalies.	Explains why a read-then-write seat check can fail under snapshot isolation; justifies a database-enforced constraint.	Pre-dates modern engines; anomalies must be mapped to PostgreSQL manually.
[3]	Ports & Grittner, 2012	Journal (PVLDB)	Implementation of serializable snapshot isolation in PostgreSQL; cost below 7% versus snapshot isolation in the reported benchmarks.	Shows that PostgreSQL can provide serializable execution at moderate cost; serialization failures must be retried.	Benchmarks are not ticketing workloads.

No.	Source	Type	Topic and finding	Use in the project	Limitation
[4]	Villamizar et al., 2015	Conference (IEEE)	Case study of the same application as a monolith and as microservices in the cloud; discusses benefits and challenges.	Early evidence for the architecture trade-off.	One framework; 2015 cloud setting.
[5]	Blinowski et al., 2022	Journal (IEEE Access)	Controlled comparison of monolithic and microservice versions in Java and C#.NET; the monolith performed better on a single machine.	Supports starting with a modular monolith for a small deployment.	Reference application is not a ticketing system; no Go implementation.
[6]	Tanadechopon & Kasemsontitum, 2023	Conference (IEEE InCIT)	Compares PHP, Python, Node.js and Go as API services; Go responded better than Node.js as the number of users grew.	Supports Go for the API layer.	Small experiment; simple API workload.
[7]	Niswar et al., 2024	Journal (IJET)	Compares REST, GraphQL and gRPC in a containerised microservice setup; gRPC had the fastest response time, followed by REST and GraphQL.	Informs the choice of API style for internal communication.	100–500 requests; internal calls only.
[8]	Elghazal et al., 2025	Conference (WEBIST)	Compares REST and GraphQL for microservices; REST showed better memory efficiency and response time under high load.	Supports REST for the public API.	Does not include gRPC.
[9]	Marieska et al. (year/venue to be confirmed)	Repository record	Compares monolithic and microservice architectures for an online ticketing system under high load: microservices were 36% faster with 71% fewer errors, but degraded when CPU usage exceeded 90%.	Closest domain match; shows overload risk of microservices.	Only the abstract was reviewed; bibliographic data must be confirmed.
[10]	Fielding, 2000	Doctoral dissertation	Defines REST architectural constraints.	Foundation of the HTTP/JSON API design.	Style, not an implementation guide.
[11]	PostgreSQL documentation	Technical documentation	Describes transaction isolation, explicit locking and unique/partial indexes.	Primary source for the seat-hold implementation.	Version-specific; must be pinned.
[12]	Martin, 2017	Book	Clean Architecture: dependency rule and layering.	Basis for the layered backend structure (also used in the baseline prototype).	Practitioner source.
[13]	OWASP ASVS 4.0.3, 2021	Standard	Verification requirements for web application security.	Source of the security checklist in NFR-04 and the verification plan.	Level selection is a project decision.

4. Comparison of existing systems

Three services used in Kazakhstan were compared against eight criteria. The comparison uses only public information; cells are marked “Yes” or “No” only when a cited source supports them, “Indirect” when a source implies the feature, and “Not verified” when no evidence was found. **Not verified is not the same as absent**: these cells must be checked by hands-on use of each service and the screenshots stored in docs/system-comparison/ before the committee defence. Chaplin Cinemas was not included because the retrieved sources only report that it was developing its own service.

Table 7 – Comparison of existing systems

Criterion	Ticketon [S1]	Kinopark [S2, S3]	Kino.kz [S4]	Proposed platform
C1 Several cinemas in one service	Yes [S1]	No — own network only [S2]	Yes [S4]	Out of scope (single operator, multi-hall)
C2 Online seat selection	Indirect [S1]	Indirect [S2]	Yes [S4]	Yes (FR-06, FR-07)
C3 Online payment	Yes [S1]	Yes [S2, S3]	Yes [S4]	Sandbox (FR-09)
C4 E-ticket on the phone	Yes [S1]	Not verified	Not verified	Yes (FR-12)
C5 Mobile application	Yes [S1]	Yes [S2]	Yes [S4]	No (responsive web)
C6 Purchase history / reminders	Not verified	Yes [S2]	Not verified	History: yes (FR-10)
C7 No extra fee for the viewer	Not verified (see [S3])	Yes, on its own site [S3]	Not verified	Not applicable (no real payments)
C8 Self-service tools for a small cinema	Not verified	No (own network)	Not verified	Yes (FR-13 – FR-17)

Sources for this section:

[S1] Ticketon.kz, “About us”, application-store listing (mirror at mergeek.com/en/latest/aBgrLW6xjlwnj0J3).

[S2] Apple App Store, “Kinopark – Kinoplexx Theatres” (CINEMA INVEST GROUP TOO), apps.apple.com/kz/app/kinopark-kinoplexx-theatres/id6467143542.

[S3] Forbes.kz, “Киносети создадут конкуренцию системе онлайн-продаж «Тикетон»”, forbes.kz/articles/kinoseti_sozdadut_konkurentsiyu_sisteme_onlayn-prodaj_tiketon (publication date to be confirmed).

[S4] Weproject.media, “17 приложений и сервисов для комфортной жизни и путешествий по Казахстану” (entry on Kino.kz).

Conclusions

The reviewed services already provide online purchase and seat choice for large audiences. They are either aggregators serving many venues or the closed system of one network [S1, S3]. None of the retrieved sources shows a self-service platform for a small independent cinema together with a documented guarantee against double booking. The proposed solution therefore does not try to compete in coverage; it targets a single operator, verifiable seat-booking correctness and an open, documented implementation. This claim about the gap is limited by the “Not verified” cells and will be re-evaluated after the hands-on check and the interviews.

5. Technology comparison

Five key technical decisions were compared, each with two or three reasonable alternatives. Ratings are qualitative and rely on the cited sources or on documented team experience (the students' justification documents list Go, Gin, gRPC and PostgreSQL for Nurbek, Java, Spring Boot, PostgreSQL and Redis for Ismail, and Next.js with TypeScript for Erasyl). The decisions are proposals to be confirmed with the supervisor.

D1. Backend platform

Table 8 – Backend platform

Criterion	Go (net/http or Gin)	Node.js (Express / NestJS)	Java (Spring Boot)
Concurrency model	Goroutines and channels	Single-threaded event loop; worker threads	Threads; virtual threads in recent JDK
Evidence on API performance	Better than Node.js as users grow [6]	Good at low load [6]	Mature; strong after warm-up [5]
Team experience	Nurbek: Go microservices, Gin, gRPC	Not documented	Ismail: Java, Spring Boot, Spring Security, JPA
Deployment footprint	Single static binary, small container	Runtime and dependencies required	JVM required, larger memory

Proposed decision: Go — to be agreed by the whole team, because the two back-end developers have experience in different stacks (Go and Java/Spring Boot). Go supports straightforward concurrent request handling and is supported by the cited comparison [6]. If the team chooses Java/Spring Boot instead, D2–D5 remain valid unchanged; only this decision and the repository layout change.

D2. Data storage

Table 9 – Data storage

Criterion	PostgreSQL	MySQL / MariaDB	MongoDB
Transactions (ACID)	Full, multi-statement	Full (InnoDB)	Multi-document transactions available, less natural for relational data
Partial unique index (one active ticket per seat and session)	Supported [11]	Not supported natively; needs workaround	Unique index only with a partial filter expression
Isolation guarantees	Read committed, repeatable read (snapshot), serializable (SSI) [3, 11]	Repeatable read by default	Snapshot within a transaction
Team experience	High (used in prototype)	Not documented	Not documented

Decision: PostgreSQL. The domain is relational (halls, seats, sessions, tickets) and the "one active ticket per seat in a session" rule can be enforced by the database itself with a partial unique index, which is the most reliable protection against the double-booking anomalies described in [2].

D3. Seat-hold mechanism

Table 10 – Seat-hold mechanism

Criterion	DB-only: transaction + partial unique index + expiry timestamp	Redis lock (SET NX EX) + DB	Optimistic version column [1]
Correctness under concurrency	Enforced by the database; independent of application code	Depends on lock handling; database constraint still needed	Correct if every writer checks the version

Criterion	DB-only: transaction + partial unique index + expiry timestamp	Redis lock (SET NX EX) + DB	Optimistic version column [1]
Additional components	None	Redis server	None
Failure modes	Expired holds must be cleaned or ignored by query	Redis outage or lock expiry before commit	Retries under high contention
Restart safety	State persisted	State split between Redis and DB	State persisted

Decision: DB-only for the first release, with an expiry timestamp and a periodic clean-up job; Redis is kept as an option if load tests (Section 9) show database contention. This also removes the in-memory ticket map used in the baseline prototype.

D4. API style for the browser client

Table 11 – API style for the browser client

Criterion	REST / JSON	gRPC	GraphQL
Browser support	Native	Needs gRPC-Web proxy	Native over HTTP
Reported performance	Better than GraphQL in memory and response time under high load [8]	Fastest response among the three in [7]	Higher CPU use in [7]
Tooling for a small team	Simple; OpenAPI	Protocol Buffers, code generation	Schema and resolvers

Decision: REST/JSON for the public API [10]; gRPC is reserved for internal calls if the system is later split into services [7].

D5. System structure

Table 12 – System structure

Criterion	Modular monolith	Microservices
Single-machine performance	Better in a controlled comparison [5]	Network overhead [5]
High load	Reference point	36% faster and 71% fewer errors in a ticketing experiment, but degraded above 90% CPU [9]
Operational complexity	One deployable unit	Several services, orchestration [4]
Fit for a student team and a single cinema operator	High	Low

Decision: modular monolith with clear module boundaries (auth, catalogue, booking, statistics) so that the booking module can be extracted later. Both architectures are acknowledged as valid; the choice is driven by team size and expected load.

6. Requirements specification

Priorities use the MoSCoW scale (Must, Should, Could, Won't). Priorities are provisional until the interview results of Section 2 are entered. Each requirement has an acceptance criterion that can be checked by an automated test, a measurement or an inspection.

6.1 Functional requirements

Table 13 – Functional requirements

ID	Requirement	Priority	Acceptance criterion
FR-01	A guest can register with name, e-mail and password.	Must	Valid data → 201 and a user record with a hashed password; duplicate e-mail → 409; invalid e-mail → 400.
FR-02	A user can log in and log out.	Must	Correct credentials → token; wrong credentials → 401; token expires after the configured time.
FR-03	Access is restricted by role (user, administrator, manager).	Must	Anonymous → 401; user on an administrator endpoint → 403; verified by automated tests for every protected route.
FR-04	A visitor can browse the movie catalogue.	Must	The list shows title, genre, duration and rating from the database.
FR-05	A visitor can view sessions of a movie by date.	Must	Only future sessions are listed, sorted by start time.
FR-06	A user can view the seat map of a session with seat statuses.	Must	Each seat is free, held or booked; statuses match the database at the time of the request.
FR-07	A user can hold up to six seats for a limited time.	Must	Held seats appear as unavailable to others; a hold expires after the configured time (default 10 minutes) and the seat becomes free again.
FR-08	A user can confirm a booking of held seats.	Must	With 100 concurrent requests for the same seat exactly one succeeds and the rest receive 409.
FR-09	A user can pay for a booking through the payment provider (sandbox).	Should	Successful payment → status BOOKED; failed or timed-out payment → hold released.
FR-10	A user can view their tickets and history.	Must	Only the owner (and administrators) can open a ticket; others receive 403.
FR-11	A user can cancel a ticket before the session starts.	Should	Cancelled ticket frees the seat immediately; a cancellation after the cut-off time is rejected.
FR-12	A ticket has a QR code or unique code for entry.	Should	The code is unique and resolves to exactly one ticket.
FR-13	An administrator can create, edit and delete movies.	Must	Changes are visible in the catalogue; deleting a movie with future tickets is rejected.
FR-14	An administrator can manage halls and seat layouts.	Must	A hall has a defined number of rows and seats; layout changes do not affect past sessions.
FR-15	An administrator can create, edit and delete sessions.	Must	A session has movie, hall, start time and price; sessions with tickets cannot be deleted without cancelling them.
FR-16	The system rejects overlapping sessions in the same hall.	Should	Creating an overlapping session returns 409.
FR-17	A manager can view sales and occupancy statistics.	Should	Tickets sold, revenue and occupancy per movie and per period agree with a manual count on test data.
FR-18	The system sends a booking confirmation by e-mail.	Could	A confirmation is sent within one minute of payment in the test environment.
FR-19	A visitor can filter movies by genre and date.	Could	Filters return the expected subset on test data.
FR-20	Ratings and reviews of movies.	Won't (this release)	Deferred; the earlier prototype includes a rating field, top-rated retrieval and statistics, but a complete user-review workflow is not established.

6.2 Non-functional requirements

Table 14 – Non-functional requirements

ID	Category	Requirement	Acceptance criterion
NFR-01	Correctness	No seat is sold twice.	0 duplicate active tickets after a test with 10,000 concurrent booking attempts on a limited set of seats.
NFR-02	Performance	Seat map and booking operations respond quickly under load.	p95 latency below 300 ms for the seat map and below 500 ms for booking at 200 concurrent virtual users on the reference machine (proposed targets, to be calibrated on the baseline).
NFR-03	Security	Credentials and secrets are protected.	Passwords stored with bcrypt or Argon2; secrets read from environment variables; repository scan finds no secrets; OWASP ASVS Level 1 checklist completed [13].
NFR-04	Reliability	State survives a restart.	Holds and tickets are stored in the database only; after a restart, all holds are still valid until expiry.
NFR-05	Usability	Booking is quick for a first-time user.	Booking completed in at most five steps; mean System Usability Scale score of at least 68 with five test users.
NFR-06	Maintainability	Code is layered and tested.	Handlers, services and repositories are separated; at least 70% statement coverage of the business-logic layer; CI runs tests on every pull request.
NFR-07	Portability	The system can be started on any machine with Docker.	A single command starts the application and the database; documented in the README.
NFR-08	Observability	Operators can see what the system is doing.	Structured logs with request ID; health endpoint returns database status.
NFR-09	Compatibility	Works in current browsers and on phones.	Layout usable at 360 px width in the latest Chrome, Firefox and Safari.
NFR-10	Documentation	The API is documented.	An OpenAPI description covers all public endpoints.

Numeric targets in NFR-02 and NFR-06 are proposals and will be calibrated on the baseline measurements (V-4).

7. Architecture and diagrams

The architecture description is consistent with the requirements in Section 6; the traceability between requirements, use cases and components is given in Table 15. Editable sources (Graphviz .dot files) and image exports are stored in the repository folder diagrams/.

7.1 System context diagram

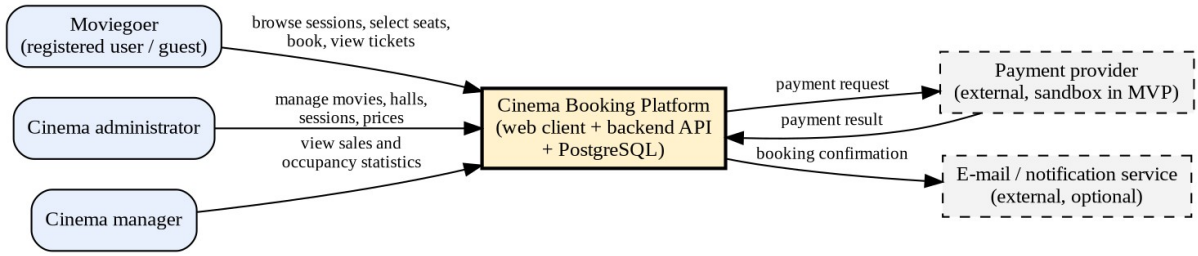


Figure 1 – System context diagram

The platform interacts with three human roles and two external systems. The payment provider is a sandbox in the first release; e-mail notification is optional (FR-18).

7.2 Use case diagram

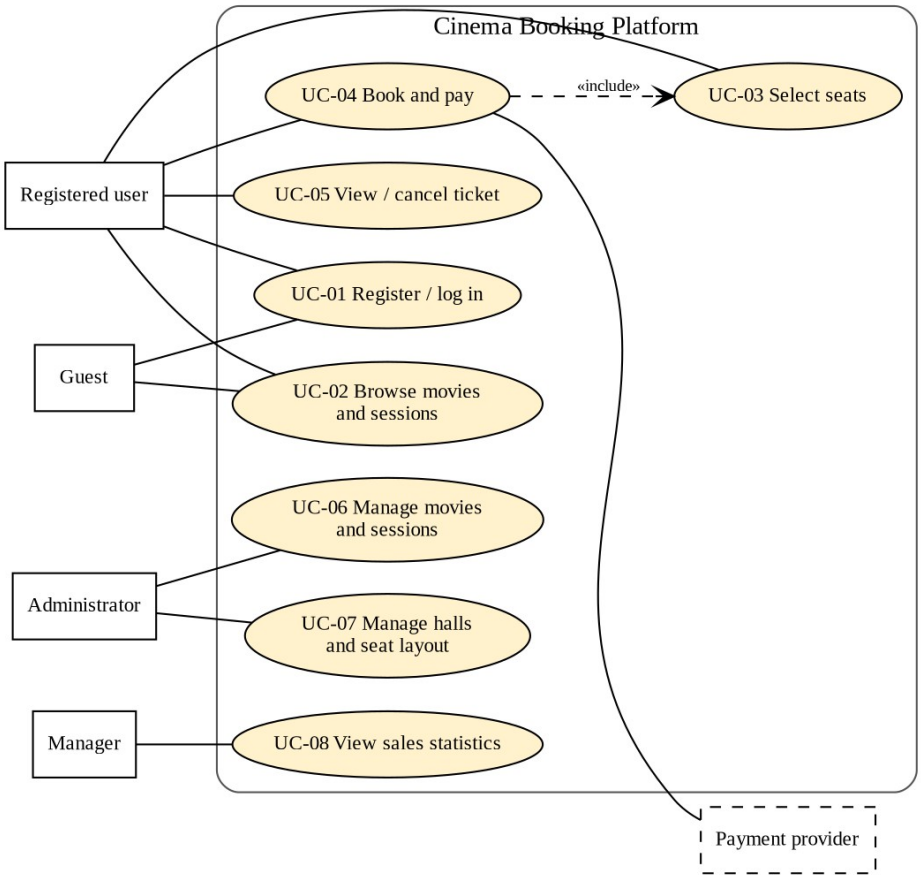


Figure 2 – Use case diagram

7.3 Component diagram (draft)

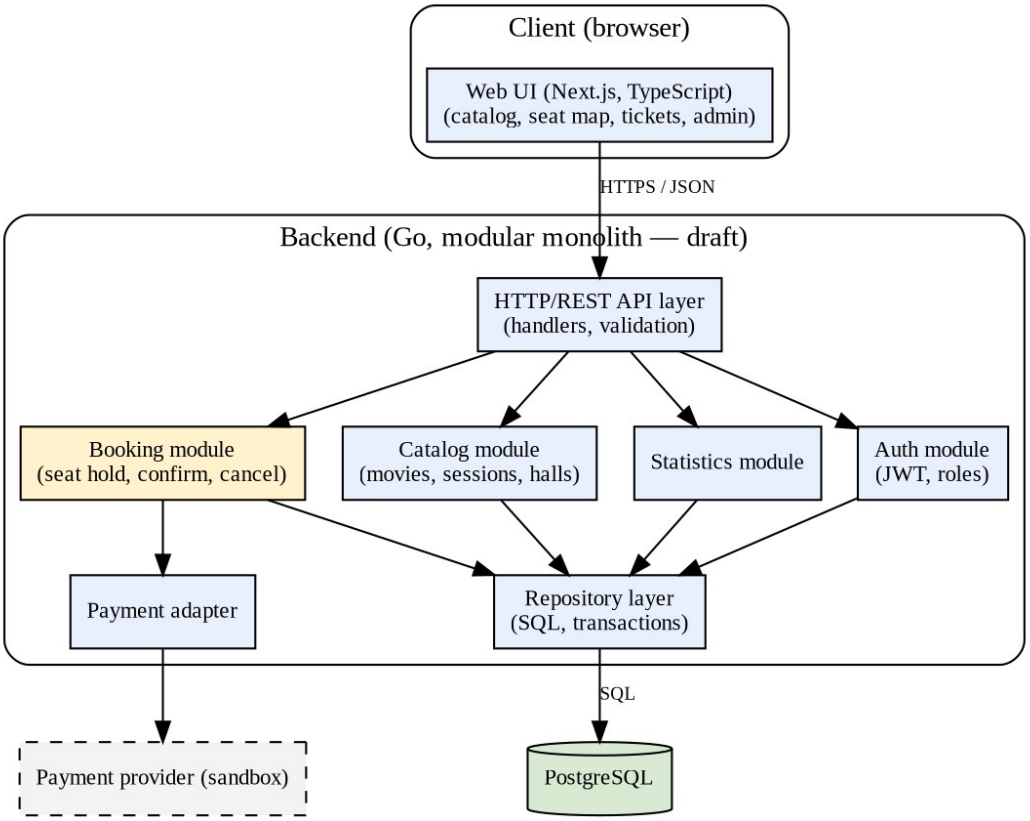


Figure 3 – Component diagram (draft)

The backend is a modular monolith. The booking module is the only module that changes seat state, and every change goes through a repository transaction. The component structure follows the layering of [12]: HTTP layer → modules (services) → repository → database.

7.4 Data model and booking life cycle

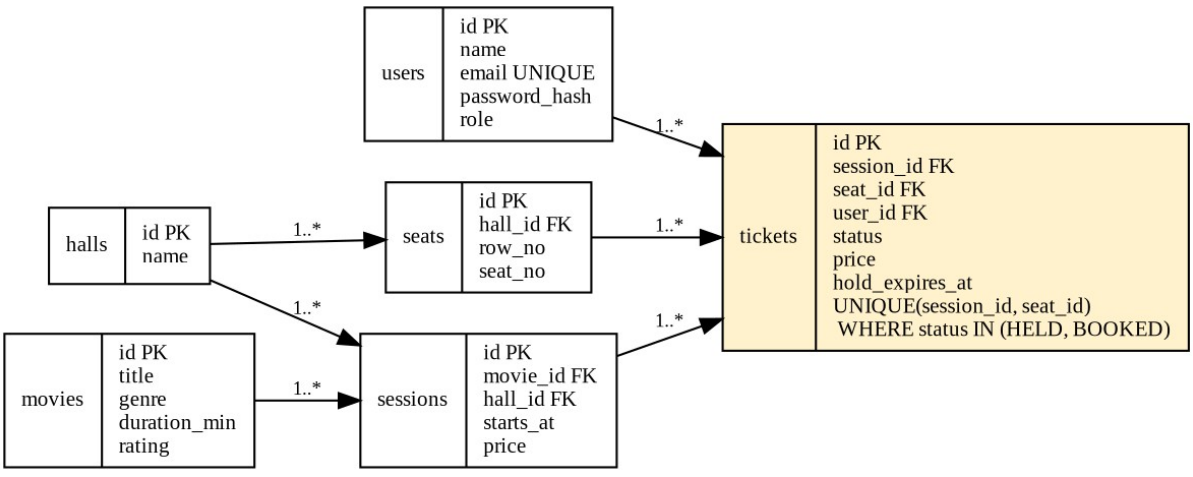


Figure 4 – Draft data model

A ticket moves through the states HELD → BOOKED → CANCELLED, or HELD → EXPIRED when the hold time ends. The database allows only one ticket in the state HELD or BOOKED for each pair (session, seat) through a partial unique index; an expired hold is ignored by availability queries and removed by a periodic job. A booking attempt for a taken seat therefore fails inside the database and the API returns 409.

7.5 Traceability

Table 15 – Requirements, components and use cases

Requirements	Components	Use cases
FR-01 – FR-03	Auth module; HTTP layer middleware	UC-01
FR-04 – FR-05	Catalogue module	UC-02
FR-06 – FR-08	Booking module; repository layer; PostgreSQL	UC-03, UC-04
FR-09	Payment adapter; Booking module	UC-04
FR-10 – FR-12	Booking module; Web UI	UC-05
FR-13 – FR-16	Catalogue module; administrator UI	UC-06, UC-07
FR-17	Statistics module	UC-08

7.6 Key architectural decisions

- Modular monolith with extractable booking module (D5, Section 5).
- REST/JSON public API with an OpenAPI description (D4).
- Seat state only in PostgreSQL; no process-local state (D3, NFR-04).
- Secrets from environment variables only (NFR-03).

8. UX prototype

Four low-fidelity wireframes cover the key scenarios: browsing sessions (S-1), selecting seats (S-2), receiving a ticket (S-3) and administering sessions (S-4). Source images and the script that generates them are in mockups/. A clickable prototype link can be added to the README if the team builds one in a design tool.

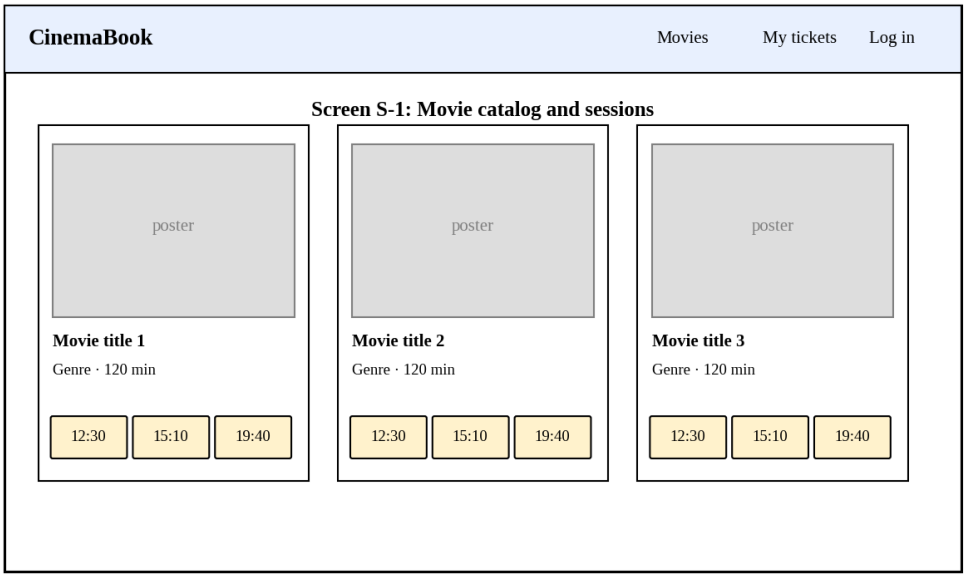


Figure 5 – Screen S-1: movie catalogue and sessions

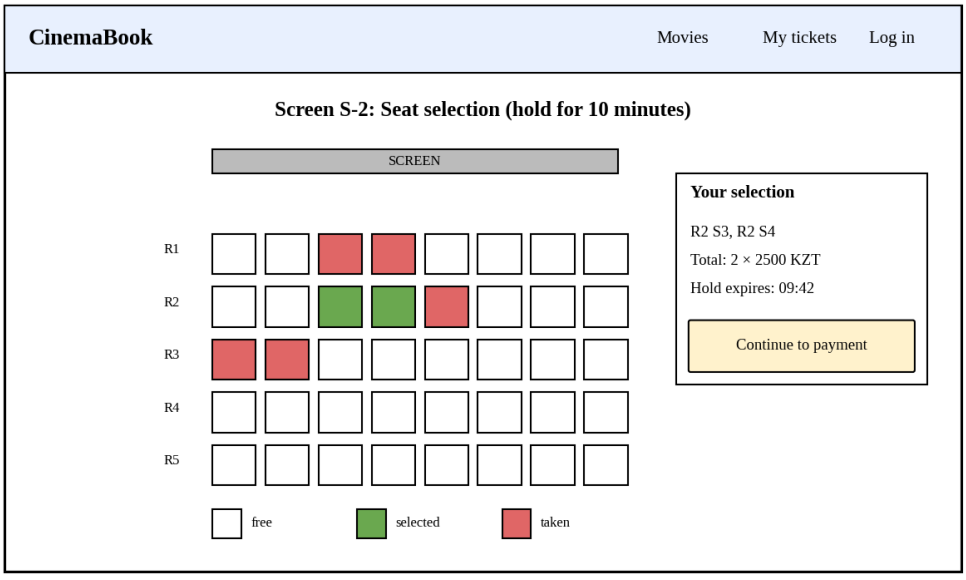


Figure 6 – Screen S-2: seat selection with hold timer

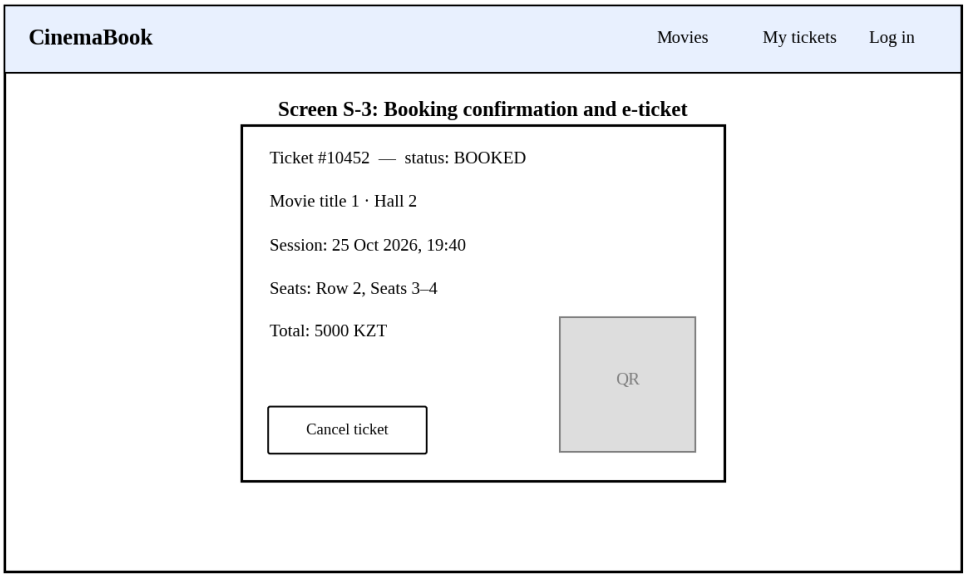


Figure 7 – Screen S-3: booking confirmation and ticket

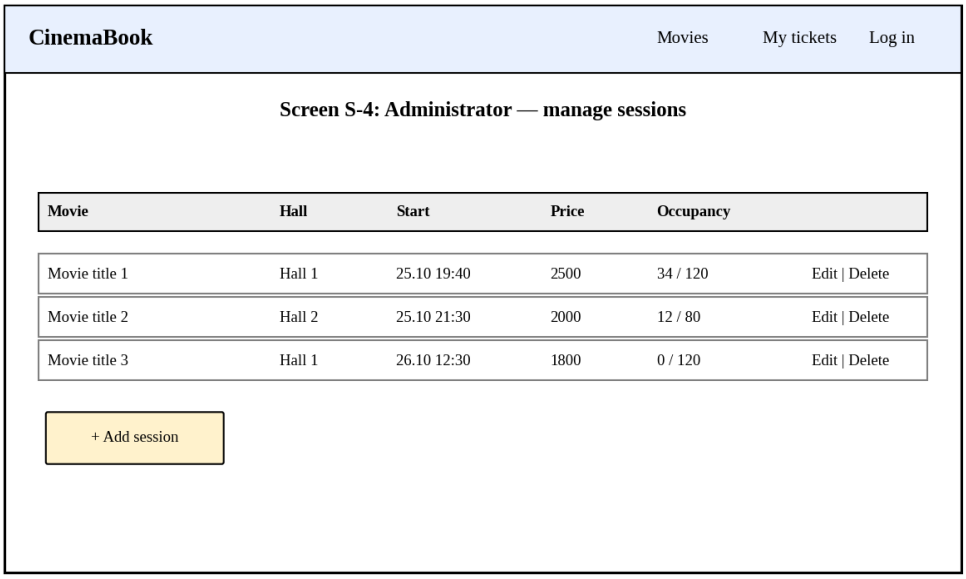


Figure 8 – Screen S-4: administrator, manage sessions

8.1 API examples

Examples show the intended interface (draft, subject to change):

```
POST /api/sessions/42/holds
{ "seat_ids": [311, 312] }
→ 201 { "hold_id": 9, "expires_at": "2026-11-02T19:50:00Z" }
→ 409 { "error": "seat_unavailable", "seat_ids": [312] }

POST /api/holds/9/confirm
→ 200 { "ticket_ids": [10452, 10453], "status": "BOOKED", "total": 5000 }
```

8.2 Previous course-project experience

Before the diploma project, the team developed a related educational cinema-booking prototype, supplied as Advanced_Final-master. This work provides practical experience with the domain and an initial implementation to analyse; it is not presented as the completed diploma platform or as evidence of validated user demand.

The supplied source snapshot uses Go with net/http, PostgreSQL access through database/sql, and an HTML/CSS/JavaScript client. It contains movie CRUD, password hashing with bcrypt, JWT-based authentication, role checks, ticket creation, owner-checked ticket retrieval and ticket history. It also includes a rating field, top-rated movie retrieval and aggregate statistics. These features gave the team experience with HTTP/JSON integration, relational data access, access control and the catalogue-to-ticket workflow.

The seven screenshots supplied by the team illustrate registration, login, a dashboard, movie management, booking, ticket history and an analytics/rating screen from the earlier demonstration. They are visual evidence of that demonstration, not proof that every feature is complete or reproducible from the supplied archive. In particular, login.html, register.html and rating.html are empty in this source snapshot. The exact version corresponding to the screenshots remains to be identified.

The prior implementation also exposes concrete design lessons. The booking handler uses a session identifier to query a movie record, while the interface asks for a numeric seat rather than presenting a hall map. The archive does not establish a persistent hold/expiry mechanism or a database constraint preventing duplicate active tickets. Credentials are embedded in the source, administrator assignment relies on an untrusted e-mail suffix, and the movie-creation route is not protected by the administrator middleware. These issues motivate explicit session/hall modelling, transactional seat allocation, secure configuration and consistent server-side authorization in the diploma design.

The diploma project therefore retains the useful domain workflow while proposing a structured web client, a proper session-seat model, time-limited holds, verified concurrency control, reproducible configuration and automated evaluation. These are planned improvements, not measured results. The preparation and limits of the intended baseline are described in Section 9.3.

Previous demonstration: selected interface screens

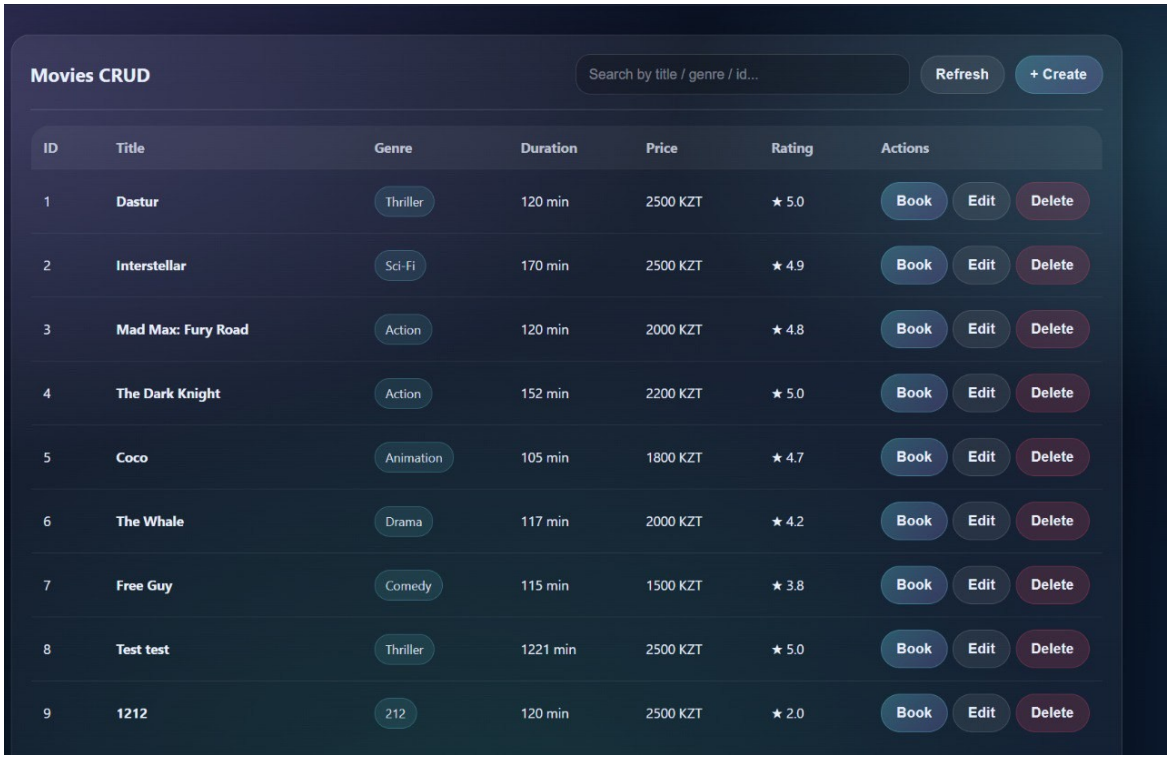


Figure P1 – Previous course prototype: movie catalogue and CRUD controls. Team-supplied demonstration screenshot; not the proposed diploma UX.

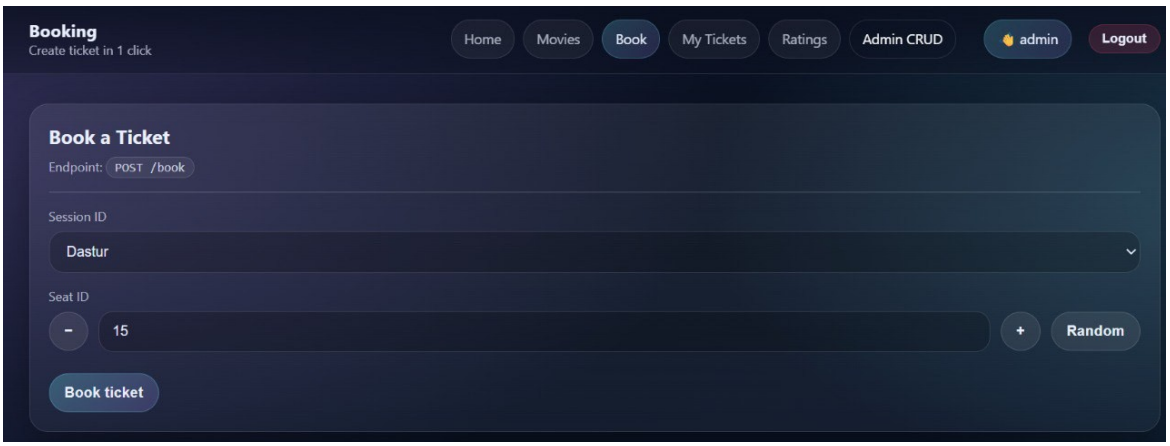


Figure P2 – Previous course prototype: booking by session/movie choice and numeric seat. A visual seat map and hold timer are proposed improvements.

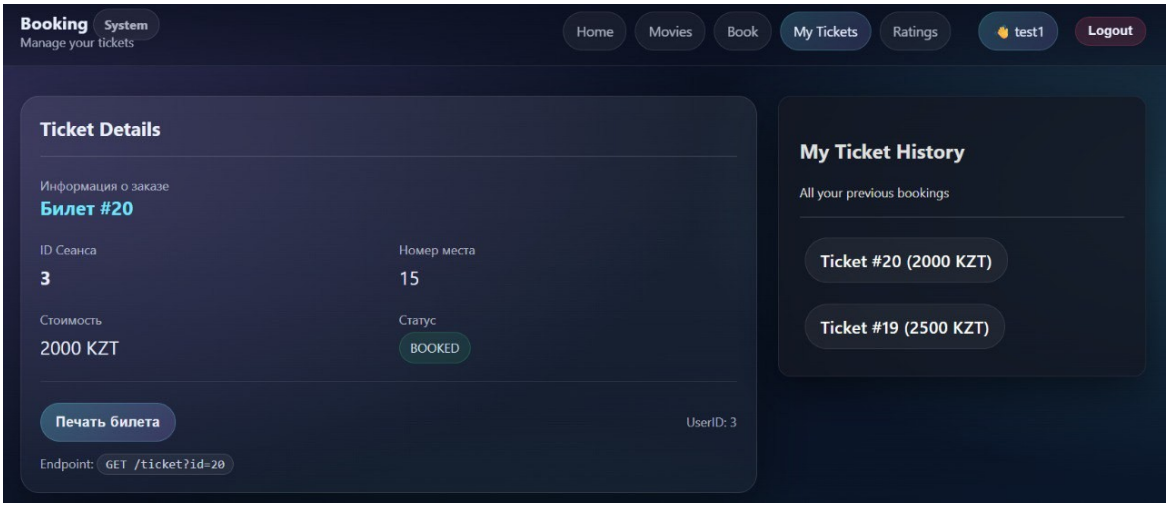


Figure P3 – Previous course prototype: ticket details and history. The displayed BOOKED status does not establish payment integration or concurrency safety.

9. Verification and evaluation plan

9.1 Tests and metrics

Table 16 – Verification plan

ID	Test	Conditions	Metrics	Success criterion	Baseline comparison
V-1	Seat-conflict test	Start N goroutines/clients that try to book the same seat at the same time (N = 100, then 10,000 attempts over 50 seats).	Number of successful bookings per seat; duplicates	Exactly 1 per seat; 0 duplicates (NFR-01)	Baseline outcome to be measured; safety is not established by static review. New solution must meet the stated success criterion.
V-2	Unit tests	Business rules: hold expiry, cancellation cut-off, price calculation, role checks.	Coverage; failing tests	At least 70% of the business-logic layer (NFR-06)	—
V-3	API integration tests	Full request flow against a test PostgreSQL in a container.	Pass rate	100% of acceptance criteria in Section 6 automated where feasible	Baseline has no tests
V-4	Load test (k6)	Ramp to 200 virtual users on seat map and booking endpoints; seed: 10 halls, 150 seats each, 200 sessions.	p50/p95 latency, error rate, throughput	p95 < 300 ms (seat map) and < 500 ms (booking); error rate < 1% (NFR-02)	Same scenario on the baseline
V-5	Restart test	Create holds, restart the application, check holds and tickets.	Holds preserved	100% preserved (NFR-04)	Baseline persists tickets in PostgreSQL and also keeps a process-local map; test durable persistence separately from cache loss.
V-6	Security checklist	OWASP ASVS Level 1 items; secret scan of the repository.	Items passed; secrets found	All applicable items passed; 0 secrets (NFR-03)	Baseline contains hard-coded credentials
V-7	Usability test	Five users perform the booking scenario on the prototype; SUS questionnaire.	Task success rate; time; SUS	SUS ≥ 68; at most five steps (NFR-05)	—

9.2 Measurement conditions

- Reference machine, PostgreSQL version, Go version and Docker image tags are recorded in the repository before each measurement.
- Seed data: 10 halls with 150 seats each and 200 sessions; the same data set is used for the baseline and the new system.
- Each load test is repeated at least three times; median and spread are reported.
- The test database is recreated before each run.

9.3 Baseline solution

The earlier course prototype (Advanced_Final) is the intended baseline. A reproducible baseline has not yet been established: the archive does not include the database schema, migrations, seed data or pinned external Go dependencies. The team must identify the original snapshot, reconcile it with the supplied screenshots, and document any configuration or reproducibility patches before measurement. Only equivalent operations should be compared; the new seat-map and hold-expiry functionality requires separate absolute tests. No baseline test outcomes are claimed here. Static review identified the following limitations:

- The database password and the token-signing key are written directly in the source code (cmd/main.go).
- The booking handler loads the price using the session identifier as a movie identifier, so sessions and movies are not separated in the data model.

- Created tickets are inserted into PostgreSQL and additionally stored in a process-local map. The map is lost on restart, but ticket retrieval reads PostgreSQL; loss of this map alone does not demonstrate loss of persisted tickets.
- The database schema is not in the repository, so protection against duplicate seat tickets could not be confirmed from the code.
- The login.html, register.html and rating.html files in the supplied source snapshot are empty, although the team-supplied screenshots show related screens. Backend code includes login, registration, top-rated movies and aggregate statistics; a complete user-review workflow is not established. The screenshots and source archive must be mapped to the same version before a reproducible demonstration. No automated tests are included in the archive.

Because the baseline source contains credentials, any live database password and token-signing key used by that source must be rotated before sharing or running it. Replace embedded values with environment configuration and review repository history for exposure. Coordinate any history cleanup with the team and preserve a traceable record of the baseline preparation.

10. Project management and risks

10.1 Repository and task tracker

- Git repository: [link] (private access granted to supervisor and committee: [yes/no]).
- YouTrack project: [link]; board and task screenshots are submitted as a separate PDF.
- Branches are named by task ID (for example DIP-5-booking-module); commits and pull requests reference the task ID; every pull request is reviewed by another team member; main is the reviewed baseline; each pre-defence is tagged (pre-defense-1, pre-defense-2, final-pre-defense).
- Repository layout: src/, docs/, diagrams/, mockups/, tests/, config/ with .env.example; README.md contains the mandatory sections.

10.2 Backlog and responsibilities

Table 17 – Prioritised tasks (initial backlog)

ID	Task	Owner	Priority	Acceptance criterion	Deadline
DIP-1	Create repository, structure, .gitignore and README	Team	Must	Repository has the required folders; README has all mandatory sections	[date]
DIP-2	Set up YouTrack project and board; create tasks	Team	Must	Every task has owner, outcome, acceptance criteria and deadline	[date]
DIP-3	Remove secrets from the baseline code; add .env.example	Nurbek Shagarbek	Must	No secrets in the repository; configuration read from environment	[date]
DIP-4	Database schema and migrations (halls, seats, sessions, tickets)	Nurbek Shagarbek	Must	Migration runs on an empty database; partial unique index in place	[date]
DIP-5	Booking module: hold, confirm, release, expiry	Nurbek Shagarbek	Must	V-1 passes with 100 concurrent attempts	[date]
DIP-6	Authentication and roles	[owner]	Must	FR-01 – FR-03 acceptance tests pass	[date]
DIP-7	Catalogue API (movies, sessions)	[owner]	Must	FR-04, FR-05, FR-13, FR-15 tests pass	[date]
DIP-8	Seat map UI and booking flow	[owner]	Must	Screens S-1 and S-2 implemented; booking in at most five steps	[date]
DIP-9	Administrator UI	[owner]	Must	Screen S-4 implemented; FR-13 – FR-15 usable	[date]
DIP-10	Payment adapter (sandbox)	[owner]	Should	FR-09 acceptance criteria met	[date]
DIP-11	Statistics module	[owner]	Should	FR-17 matches manual count	[date]
DIP-12	Automated tests and CI	[owner]	Must	CI runs unit and integration tests on every pull request	[date]
DIP-13	Load test scripts and baseline measurements	[owner]	Must	V-4 results recorded for baseline and new system	[date]
DIP-14	OpenAPI description	[owner]	Should	All public endpoints documented	[date]
DIP-15	User interviews and findings summary	Team	Must	3–5 interviews documented in Section 2	[date]
DIP-16	Usability test and SUS evaluation	[owner]	Should	V-7 results recorded	[date]

Owners shown as [owner] and deadlines shown as [date] must be assigned by the team; the same tasks are provided as a CSV file for import into YouTrack.

10.3 Milestones

Table 18 – Milestones

ID	Milestone	Date
M1	Approbation submission: proposal, analysis, requirements, architecture, prototype, verification plan	24 October 2026
M2	Stage 1 complete: requirements confirmed with the supervisor; schema and booking module with V-1 passing	[date]
M3	Stage 2: API endpoints, authentication, catalogue and database operations complete	[date]
M4	Stage 3: load testing, optimisation, front-end integration	[date]
M5	Stage 4: final debugging, explanatory note, defence materials	[date]

The stages correspond to the four stages of the supervision plan: requirements and architecture; core backend and database; load testing and integration; final debugging and defence materials.

10.4 Risks

Table 19 – Risk register

ID	Risk	Probability	Impact	Mitigation	Owner
R-1	Interviews are not completed in time, so priorities rest on assumptions.	Medium	High	Schedule interviews immediately; mark unverified items as assumptions.	Team
R-2	Concurrency defects in the booking module.	Medium	High	Database-enforced constraint; V-1 run in CI.	Nurbek Shagarbek
R-3	Unequal workload or missing evidence of individual contributions.	Medium	Medium	Task per student in YouTrack; pull requests reviewed by another member.	Team
R-4	Payment provider integration is unavailable.	Medium	Low	Use a simulated provider behind an adapter interface.	[owner]
R-5	Secrets are committed to the repository.	Low	High	Secret scan; .env.example only; rotate the credentials found in the baseline.	Nurbek Shagarbek
R-6	Access to repository or YouTrack is not granted to the committee.	Low	High	Check access with a second account before submitting the form.	Team
R-7	Database contention under load.	Low	Medium	Index review; optional Redis hold layer.	Nurbek Shagarbek
R-8	Scope growth (loyalty, mobile app).	Medium	Medium	Scope fixed in Section 1; changes go through the supervisor.	Team

10.5 Dependencies

- Agreement on a single back-end stack (Go or Java/Spring Boot) before implementation starts.
- Access to a PostgreSQL instance (Docker) on every developer machine.
- Availability of the supervisor for weekly meetings and code review.
- Access to cinema staff or representatives for interviews.
- A sandbox payment provider or an agreed simulation.

11. Supervisor feedback

Feedback on key deliverables and the team's response must be recorded here after each review with the supervisor. **No feedback has been recorded yet; the table below must be completed with real comments before submission.**

Table 20 – Supervisor feedback and responses

Date	Deliverable	Supervisor comment	Decision / change made	Evidence (commit, PR, task)
[date]	Proposal and scope	[comment]	[response]	[link]
[date]	Requirements and architecture	[comment]	[response]	[link]
[date]	Technology choices	[comment]	[response]	[link]
[date]	Verification plan	[comment]	[response]	[link]

References

[1] Kung H. T., Robinson J. T. On optimistic methods for concurrency control // ACM Transactions on Database Systems. 1981. Vol. 6, No. 2. P. 213–226.

[2] Berenson H., Bernstein P., Gray J., Melton J., O'Neil E., O'Neil P. A critique of ANSI SQL isolation levels // Proceedings of ACM SIGMOD. 1995.

[3] Ports D. R. K., Grittner K. Serializable snapshot isolation in PostgreSQL // Proceedings of the VLDB Endowment. 2012. Vol. 5, No. 12. P. 1850–1861. DOI: 10.14778/2367502.2367523.

[4] Villamizar M., Garcés O., Castro H., Verano M., Salamanca L., Casallas R., Gil S. Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud // IEEE, 2015.

[5] Blinowski G., Ojdowska A., Przybyłek A. Monolithic vs. microservice architecture: a performance and scalability evaluation // IEEE Access. 2022. Vol. 10. DOI: 10.1109/ACCESS.2022.3152803.

[6] Tanadechopon T., Kasemsontitum B. Performance evaluation of programming languages as API services for cloud environments: a comparative study of PHP, Python, Node.js and Golang // Proceedings of InCIT. IEEE, 2023. P. 293–297. DOI: 10.1109/InCIT60207.2023.10413079.

[7] Niswar M., Safruddin R. A., Bustamin A., Aswad I. Performance evaluation of microservices communication with REST, GraphQL, and gRPC // International Journal of Electronics and Telecommunications. 2024. Vol. 70, No. 2. P. 429–436. DOI: 10.24425/ijet.2024.149562.

[8] Elghazal M. S. M., Aneiba A. et al. Performance evaluation of REST and GraphQL API models in microservices software development domain // Proceedings of WEBIST 2025. SciTePress, 2025. P. 83–91.

[9] Marieska M. D., Yunanta A., Aulia H., Utami A. S., Rizqie M. Q. Performance comparison of monolithic and microservices architectures in handling high-volume transactions. [Year and venue to be confirmed; repository record at repository.horizon.ac.id/items/show/10524].

[10] Fielding R. T. Architectural styles and the design of network-based software architectures: PhD dissertation. University of California, Irvine, 2000.

[11] PostgreSQL Global Development Group. PostgreSQL documentation: Concurrency control; Indexes. [Version to be pinned in the README].

[12] Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

[13] OWASP. Application Security Verification Standard 4.0.3. 2021.

Web sources used in Section 4 are listed there as [S1]–[S4].